

cloudtorch / develop — Step 4a outline

The step-4 space-time PINN, unchanged, trained on the *fixed given* background of step 3a instead of the fitted PCA background — the **given-background sibling** of step 4, and the *intrinsically well-posed* member of the pair. **Reads as a delta against step 4:** only what differs is stated here; the shared field, physics and loss are exactly as in `step_4_outline`.

1. OBJECTIVE (THE DELTA)

Keep the step-4 neural field $f_\theta(x, y, t)$ (Fourier features + MLP, trained *through* the two-cloud physics) *exactly as is*, but stack it on the **pre-calculated fixed** background $I_{\text{in}} = \text{bdata} = \text{new_fit} - \text{new_extrafit}$ of step 3a rather than the fitted PCA background of step 4. The field then predicts **only the free cloud numbers** (from $S, \Delta\tau, v_{\text{los}}, \Delta v, \log a$ per cloud — any of which may be frozen to its init, see §3); the background is *supplied, not learned* $\Rightarrow$ **0 background DoF**. Because the background is fixed, the background/cloud degeneracy that step 4 still shows on a minority of pixels **cannot arise by construction** — this is the well-posed PINN.

2. WHAT CHANGES VS STEP 4 (THREE EDITS; EVERYTHING ELSE IDENTICAL)

- **Flexible output width** $n_{\text{out}} = n_{\text{bkg}} + n_{\text{free}}$, not a fixed 14. The *given* mode sets $n_{\text{bkg}} = 0$ (no background columns); n_{free} is the count of *un-frozen* cloud parameters (§3). Fully free $\Rightarrow 8$; freezing any parameter shrinks the head further.
- **Route through the given forward.** Feed the field's `p_cloud` to `composite_synth_given(xl, I_in, p_cloud)` instead of `composite_synth(xl, pca, c, p_cloud)`. The coefficient slice `c` is empty and unused; *no* PCA is loaded and *no* background bias anchoring is needed.
- **Supply I_{in} .** Load it once with step 3a's `load_given_background(cfg, meta)` — aligned to the same window and frames `ts`, `grad-free`; its row order already matches `grid_coords(t, y, x)`, so it drops in beside `obs_pt`.

3. FLEXIBLE OUTPUT: FREEZE ANY PHYSICAL PARAMETER

The head width is *spec-driven*. For each of the 10 cloud slots ($S, \Delta\tau, v_{\text{los}}, \Delta v, \log a$ per cloud) a *freeze* set decides *free* vs *frozen*: a **free** slot emits one pre-activation column and passes through its range transform; a **frozen** slot emits *nothing* and is inserted as its (spatially/temporally *uniform*) init constant — exactly the mechanism the model already uses for $\log a$, now generalised to *any* parameter. So $n_{\text{free}} \in [0, 10]$, $n_{\text{out}} = n_{\text{bkg}} + n_{\text{free}}$, and the assembled `p_cloud` is always the full $(N, 10)$ the physics needs. Default freeze = $\{\log a_1, \log a_2\}$ ($\Rightarrow n_{\text{free}} = 8$); freeze e.g. a v_{los} or $\Delta\tau$ to hold it at its init everywhere and let the field solve for the rest.

4. UNCHANGED FROM STEP 4 (REUSED VERBATIM)

Fourier features with the two bandwidths (σ_{xy}, σ_t) as the space-time regulariser; MLP ($W \sim 128\text{--}256$, $D \sim 4$, SiLU); the affine output head **anchored at the step-3 cloud inits** (a frozen slot simply *is* that init); the range transforms on the free slots ($S = S_{\text{max}}\sigma(\cdot)$, $\Delta\tau = \text{softplus}$, $\Delta v = \text{dv_floor} + \text{softplus}$, v_{los} identity); loss = mean per-pixel χ^2 over the sampled (x, y, t) ; Adam on θ . `chi2_per_pixel` is reused unchanged. In short: **same network, same physics tail, same loss** — only the background source and the output width differ.

5. KNOBS = THE TWO BANDWIDTHS (SIMPLER HERE)

Scan (σ_{xy}, σ_t) as in step 4 (sweet spot there $\sigma_{xy} \approx 4$, $\sigma_t = 2$). With *no* background DoF to trade against, there is nothing for the field to cancel: fewer failure modes, and the degeneracy guardrails steps 5–6 needed (`no_emission` cap, `background_anchor`) are *moot*. The head width $n_{\text{bkg}} + n_{\text{free}}$ carries *both* axes of flexibility, so the *same* module also serves the variable- K PCA fit mode later ($n_{\text{bkg}} = K$: 1 = mean only, 2 = +1, ...) — that wiring is the step 4/6 integration, not here.

6. SCALING THE BANDWIDTHS WITH CROP / N_T

`grid_coords` normalises *every* window to $[-1, 1]^3$, so a fixed (σ_{xy}, σ_t) counts oscillations across the *whole field*, not per pixel. To hold the *per-pixel* / *per-frame* smoothness fixed when you resize the window, scale the bandwidths **linearly** with the pixel/frame count, relative to the reference (`crop_ref = 40`, $\sigma_{xy} = 4$; `n_t_ref = 4`, $\sigma_t = 2$):

$$\sigma_{xy} = 4.0 \frac{\text{CROP}}{40} = 0.1 \text{ CROP}, \quad \sigma_t = 2.0 \frac{N_T}{4} = 0.5 N_T.$$

This is exactly what `fit_cube.sigmas_for` applies for the server run. Keeping σ *fixed* while enlarging the window instead makes the maps *smoother* (fewer features per pixel). Secondary knobs do *not* track window size: raise `N_FREQ` only if you push σ much higher; `WIDTH/DEPTH` set model capacity; set `BATCH` and more `N_ITERS` as the sample count $N_t C_y C_x$ grows.

7. TEST LOCALLY, THEN THE SERVER (AS STEP 4)

Same tiny-instance-first discipline: small crop ($\sim 24 \times 24$), 3–5 frames around $t = 13$, config-driven scale, minibatch over coordinates, device `cpu` $\rightarrow$ `cuda` with no code change. Validate the full path (Fourier $\rightarrow$ MLP $\rightarrow$ `composite_synth_given` $\rightarrow$ loss $\rightarrow$ backward $\rightarrow$ step: shapes, grads reach the **free** cloud outputs only (none to frozen slots), NaN-safety, loss drops), then **compare on the identical crop** against (a) the step-4 fitted-background PINN — expect similar χ^2 but *no* degenerate pixels; and (b) the step-3a per-pixel given fit — expect similar χ^2 with *smoother* space-time maps.

8. MODULE & NOTEBOOK (AFTER APPROVAL)

Minimal `pinn_model.py` additions: a clean *given* entry ($n_{\text{bkg}} = 0 \Rightarrow$ forward returns `p_cloud`; the caller uses `composite_synth_given`) and the *freeze*-set generalisation (§3) replacing the hardcoded $\log a$ freeze, reusing step 3a's `load_given_background`. A step-4a notebook defaulting to the laptop smoke test — mirroring `step_3a_given_test` for the data/background loading and the step-4 notebook for the field and training loop.

Plan only — no code until you approve. Then the small `pinn_model.py` given-mode entry + a step-4a notebook (tiny-instance default). The full-cube run reuses it on the server: enlarge the config, `device=cuda`, same code.